

HelixTrack

HelixTrack

بديل للعالم الحر JIRA

الملخص

وبواسطة امتداده "المستندات"، JIRA هو بديل شامل وحديث ومفتوح المصدر لـ HelixTrack أيضاً — نظام متعدد المنصات لإدارة المشاريع وتتبع المشكلات، مبني على خلفية Confluence للخدمات المصغرة مع عملاء أصليين للويب و سطح المكتب والهواتف المحمولة Go.

وصف مختصر

للخدمات المصغرة Go توفر خلفية JIRA/Confluence بديل مفتوح المصدر لـ موحدة لتتبع المشاريع والمشكلات، بالإضافة إلى REST API واجهة ("HelixTrack Core") تُقدّم للعملاء الأصليين للويب و سطح المكتب وأنظمة Confluence، مساحة عمل وثائقية على نمط HTTP/3 QUIC عبر iOS و Android.

وصف مفصل

هو منصة مفتوحة المصدر لإدارة المشاريع وتتبع المشكلات، تُقدّم كبديل للعالم الحر لـ HelixTrack، بديل كامل للمنتجين اللذين تعتمد عليهما معظم المؤسسات الهندسية — JIRA و Confluence **HelixTrack** أعيد بناؤهما كبرنامج تملكه ويمكن تشغيله في أي مكان. في قلب النظام يوجد، توفر تتبعاً كاملاً للمشكلات، Gin باستخدام إطار عمل Go خدمة مصغرة مكتوبة بلغة **Core**، لوحات أجايل وسكرم، إدارة الفرق، ونظام أذونات هرمي يمكن تبديل تنفيذه بين محرك داخلي محلي مما يسمح لنموذج التفويض نفسه بالتوسع من حاسوب محمول واحد إلى — HTTP وخدمة مدعومة بـ عبر عشرات المسارات، يوجّه REST مجموعة موزعة دون تعديل كود التطبيق. بدلاً من نشر واجهة موجهة بالإجراءات، مع نموذج طلب/استجابة موحد /do كل شيء، عبر نقطة نهاية واحدة Core للخارج): كل errorCode/errorMessage/data، للداخل (action/jwt/object/data) جديداً لتوثيقه وتأمينه URL عميل يتحدث نفس العقد الصغير، وإضافة ميزة تعني إضافة إجراء، وليس وإصداره.

HTTP/3 مع خدمات مصممة لفصل المصادقة والأذونات والترجمة، تتواصل عبر Core يتكامل ويمكن تشغيلها على أجهزة أو مجموعات منفصلة، أو تعطيلها بالكامل في بيئات الاختبار QUIC للإنتاج، وتُشفّر PostgreSQL لتطوير سهل وبدون إعدادات، وفي SQLite تُخزن البيانات في بحيث تُحمى بيانات المشاريع الحساسة SQLCipher (AES-256) أثناء التخزين باستخدام من تحويل المتتبع إلى Documents V2 على القرص افتراضياً وليس كإجراء لاحق. يمكن امتداد

تتضمن مساحات وصفحات ومراقبة Confluence منصة معرفية كاملة: مساحة عمل على نمط وتحليلات — حيث يلتقي الويكي ومتتبع WebSocket الإصدارات وقوالب وتعاوناً فورياً عبر المشكلات أخيراً خلف خلفية واحدة بدلاً من منتجين مدمجين معاً.

+ Tauri وعميل سطح مكتب Angular، توجد عدة تطبيقات عملاء: عميل ويب Core حول بالإضافة إلى عملاء iOS (Swift) وAndroid (Kotlin) وتطبيقات أصلية لنظامي Angular، وحافظة شاشة — كلها تتواصل مع نفس الخلفية وتكتشفها Aurora OS وHarmonyOS لأنظمة بحيث يجد العميل الجديد خادمه دون الحاجة إلى UDP، تلقائياً على الشبكات المحلية عبر بث تهيئة يدوية. تُدار تطبيقات العملاء كمستودعات منفصلة وخاصة وتُعرض هنا على مستوى المنتج فقط.

المحتوى

لماذا أنشأناها

JIRA + Confluence لنقدم للفرق بديلاً مفتوح المصدر وقابلاً للاستضافة الذاتية حقاً عن حزمة من أجل العالم الحر — دون قيود البائعين، يجمع بين تتبع المهام والمستندات والتعاون — المؤسسي تحت ترخيص مفتوح واحد.

لماذا تُعدُّ ثورة حقيقية

إنها تدمج منتجين تجاريين ثقلين — تتبع المشكلات وحزمة الويكي/المستندات — في منصة واحدة مفتوحة وعالية الأداء وقابلة للاستضافة الذاتية، وتضيف إليها ما لم تقدمه الحلول الحالية قط: عملاء أصليون حقيقيون عبر منصات متعددة (الويب، سطح المكتب، أندرويد، أي أو إس، بالإضافة إلى جميعها مدفوعة بعقد خلفي واحد. المفتاح هنا هو الملكية دون (Aurora OS وHarmonyOS SQLCipher اللامركزية بالكامل مع خدمات مصغرة منفصلة، وتشفير HTTP/3 تنازلات. تصميم أثناء التخزين، يوفران الأداء والأمان اللذين عادةً ما يقتصران على الخدمات السحابية AES-256 الخاصة، لكن في نظام تستضيفه بنفسك — دون تراخيص مقاعد، ودون قيود البائعين، ودون مغامرة التي تعرفها بالفعل JIRA-plus-Confluence بياناتك لبنيتك التحتية. تحصل الفرق على تجربة على أجهزتها الخاصة، وتحت ترخيص مفتوح المصدر واحد.

ما الجديد في الابتكار

- الموحدة القائمة على الإجراءات — نقطة نهاية واحدة، حاوية واحدة، توجيه بناء API do واجهة جديدة، مما يقلص سطح URL على الإجراءات. تصل القرات الجديدة كأفعال جديدة، لا عناوين الهجوم، وكود العميل، وأعباء التوثيق إلى عقد واحد تشترك فيه جميع المنصات.
- كنقل افتراضي بين الخدمات — شبكات حديثة منخفضة الكمون ومرنة في HTTP/3 QUIC. الاتصالات بين الخدمات منذ اليوم الأول، لا تُضاف لاحقاً.
- إلى جانب HTTP، محرك أذونات قابل للتبديل بين تنفيذ محلي داخل العملية وخدمة مدعومة بخدمات المصادقة والأذونات والتوطين القابلة للنشر بشكل مستقل — نفس نموذج التفويض سواء كنت تشغل عملية واحدة أو مجموعة خوادم.
- يحصل كل مشروع على space-root - عزل البيانات عبر مساحات متعددة باستخدام علامة قاعدة بيانات ومخزن أصول معزولين، بحيث تُفصل المستأجرون والمشاريع عند حدود التخزين بدلاً من استخدام عوامل التصفية في الاستعلامات.
- أثناء التخزين — حماية بيانات المشاريع الحساسة على AES-256 SQLCipher تشفير القرص بشكل شفاف، افتراضياً.
- على الشبكات المحلية — يكتشف UDP الاكتشاف التلقائي للعميل إلى الخادم عبر بث دون أي تهيئة يدوية Core العميل.

- مع تحرير متوازٍ بتقنية القفل المتفائل، وكشف Confluence، بديل حقيقي لـ V2، المستندات التعارضات، وسجل كامل للتغييرات — مستندات تعاونية حقيقية تعيش خلف نفس الخلفية مع أداة التتبع.

أكبر التحديات التقنية وكيف تغلبنا عليها

- الحفاظ على عملاء الويب .ست منصات عميل، خلفية واحدة، انحراف صفري في العقد / HarmonyOS و Swift/وآي أو إس Kotlin/وأندرويد Tauri/وسطح المكتب Angular متباينة تتآكل بشكل غير متزامن. لقد ألغينا API يعني عادةً ست عمليات تكامل Aurora و — القائمة على الإجراءات وحاويتها الثابتة العقدة الوحيدة API /do هذا الخطر بجعل واجهة حتى يتمكن UDP يستهدفها كل عميل بنفس الطريقة — وأضفنا فوقها اكتشاف الخدمة عبر بث على الشبكة دون تهيئة يدوية للنقاط النهائية Core العملاء من تحديد موقع تقسيم خدمات المصادقة والأدوات والتوطين إلى .فصل الخدمات دون دفع ضريبة الكمون HTTP/3 خدمات قابلة للنشر بشكل مستقل يضيف عادةً قفزة شبكية لكل استدعاء. اعتمدنا عبر جميع الاستدعاءات بين الخدمات للحفاظ على سرعة هذه القفلات ومرونة QUIC الاتصالات، وجعلنا كل خدمة قابلة للتشغيل بشكل مستقل — حتى تعطيلها بالكامل في تكوينات الاختبار — بحيث يكون الفصل خيار نشر، لا تكلفة ثابتة.
- التحرير المتعدد المؤلفين في .دون فوضى الكتابة المفقودة Confluence التعاون بمستوى /هذه المشكلة بمساحات V2 الوقت الفعلي يفتح الباب أمام تعارضات الكتابة. تحل المستندات صفحات/إصدارات تحت قفل متفائل بالإضافة إلى كشف التعارضات الصريح، وسجل كامل في الوقت الفعلي — تعاون يظل متسقاً بدلاً من WebSocket للتغييرات للعودة إليه، ومزامنة محو التعديلات بصمت.
- البيانات على SQLCipher AES-256 يحمي .التشفير أثناء التخزين دون قتل الإنتاجية في (LRU) القرص لكنه يضيف عبئاً لكل استعمال؛ عوضاً ذلك بالتخزين المؤقت متعدد الطبقات في خدمة التوطين) بحيث تظل المسارات الساخنة مثل عمليات البحث Redis الذاكرة أمام متعددة اللغات سريعة بينما تبقى البيانات مشفرة.

المحتوى

مجموعة التقنيات

- عالية الإنتاجية ومنخفضة الكمون مع إمكانية HTTP اختيرت لبناء خدمات — Go + Gin وجهاز، JWT/CORS وبرنامجهما الوسيط، REST API النشر عبر ثنائي واحد؛ تتضمن نواة الذي يتصدر النظام بأكمله /do التوجيه.
- HTTP/3 QUIC ،اختير كوسيلة نقل بين النواة وخدمات المصادقة والأدوات والتوطين — QUIC حيث يقلل تصميم المتعدد القنوات والمهاجر للاتصالات من زمن الاستجابة الطويل QUIC حيث يقلل تصميم TCP. ويتحمل الروابط غير المستقرة التي تتعطل فيها.
- نموذج علائقي واحد يدعم مخطط — (التطوير) SQLite / (الإنتاج) PostgreSQL التطوير المحلي خالياً من SQLite التتبع والمستندات الكبير عبر كلا المحركين: يبقى الإنتاج بمقياس واسع عبر ملف تكوين Postgres الإعدادات ويعتمد على الملفات، بينما يتولى production مخصص.
- SQLCipher (AES-256) اختبار للتشفير الشفاف على مستوى قاعدة البيانات أثناء السكون، مما يحمي البيانات الحساسة للمشاريع دون الحاجة إلى تشفير على مستوى التطبيق أو تغيير طريقة كتابة الاستعلامات.
- Redis داخل خدمة LRU اختير كطبقة ذاكرة تخزين مشتركة خلف ذاكرة تخزين مؤقتة — Redis التوطين، مما يوفر طبقة تخزين مزدوجة تحافظ على سرعة عمليات البحث متعددة اللغات حتى مع عبء التشفير تحتها.

- **Uber Zap + Lumberjack** — اختيرا لتسجيل البيانات المنظم وخفيف التخصيص مع دوران تلقائي للسجلات، مما يحافظ على قابلية مراقبة النواة في الإنتاج دون نمو غير محدود للسجلات.
- **golang-jwt / JWT** — اختيرا كآلية مصادقة عديمة الحالة؛ يحمل الرمز المميز الموقع في `do`، بكل مطروف `jwt` حقل.
- **Angular 19 (+ Material. RxJS)** — اختير لبناء عميل متصفح تفاعلي قائم — ناضج جاهز للاستخدام مباشرة Material على المكونات مع نظام تصميم.
- **Tauri 2.0 + Rust + Angular** — اختير لإصدار واجهة سطح مكتب أصلية بحجم — Rust داخل عرض ويب مدعوم بـ Angular صغير عبر إعادة استخدام واجهة بدلاً من تضمين Rust داخل عرض ويب مدعوم بـ Angular صغير عبر إعادة استخدام واجهة بيئة متصفح كاملة.
- **Kotlin (أندرويد) / Swift + SwiftUI (آي أو إس)** — اختيرا لتقديم عملاء أصليين — ومتوافقين مع منصات الهواتف بدلاً من عرض ويب مغلف.
- **Docker / Docker Compose (Podman متوافق مع)** — اختير للنشر القابل — Podman والتوافق مع `health`، للتكرار والحاوي عبر الحاويات مع تفعيل فحوصات بحيث لا Podman والتوافق مع `health`، للتكرار والحاوي عبر الحاويات مع تفعيل فحوصات.
- **Testify (Go)، Cypress/Playwright/Karma+Jasmine (العملاء)** — اختير للاختبار الآلي متعدد الطبقات الذي يغطي عقد الواجهة الخلفية وواجهات المستخدم — للعملاء بشكل مستقل، بما يتوافق مع بنية نواة واحدة وعدة عملاء.

ملاحظات الحالة والشفافية

- مصغرة عاملة؛ يُوثَّق امتداد REST API هي خدمة HelixTrack نواة. الحالة: تجريبية بأنه مكتمل بنسبة 95% تقريباً مع وجود مشكلة معروفة في تعيين حقول قاعدة V2 المستندات البيانات، لذا لا يُقدَّم على أنه مكتمل تماماً.
- `core/LICENSE` لكن ملف MIT، إلى رخصة `CLAUSE.md` يشير ملف. الرخصة: لم تُحدد بعد يجب حل هذا التناقض قبل تحديد الرخصة نهائياً — Apache 2.0 الرسمي هو للمشروع (مثل +50,000 طلب في الثانية، لُزمنة README الأرقام الأداء المذكورة في ملف استعلام أقل من ميلي ثانية) هي أهداف تصميم وتسويق، وليست معايير أداء منشورة بشكل مستقل، لذا تم استبعادها من الادعاءات أعلاه.
- Aurora، HarmonyOS، الويب، سطح المكتب، أندرويد، آي أو إس تطبيقات العملاء. وُثِّف فقط على مستوى المنتج خاصة موجودة في مستودعات.

المحتوى

تأتي في المرتبة الأولى — Helix-Track الرئيسية رائد خط إنتاج Helix-Track: الفئة ذات الأولوية Server Factory قبل أي مشاريع من فئة.